Database Design and Jury Preparation Handbook

Exam Scheduling and Management System

Gautam Buddha University Examination Operations

Prepared from the current application schema

How to Use This Handbook

This handbook explains the database in presentation-ready language. Read Sections 1 to 7 to understand the design, use Sections 8 to 11 during demonstration and troubleshooting, and revise the question bank before the jury. The current implementation contains 35 tables and 73 declared foreign keys. Its central design decision is to keep curriculum and scheduling separated by programme, admission batch, semester and section.

- For a five-minute explanation, study the executive summary, core relationship chain and scheduling data flow.
- For a technical jury, focus on keys, normalization, constraints, transactions, indexes and deletion behaviour.
- For a live demonstration, follow the recommended data-entry order and verification queries.
- For deployment, import schema.sql before seed.sql and never treat seed.sql as a production backup.

Contents

1. Database Overview

2. Relational Design Concepts

3. Complete Table Dictionary

4. Core Entity Relationships

5. Automatic Scheduling Data Model

6. Seating Invigilation and Attendance

7. Integrity Normalization and Performance

8. Database Installation and Maintenance

9. Demonstration and Verification Queries

10. Troubleshooting Guide

11. Jury Question Bank

12. Appendix A Complete Foreign Key Matrix

13. Appendix B Presentation Script and Revision Checklist

1 Database Overview

The database supports the complete examination workflow rather than only storing a date sheet. Academic master data defines schools, departments, programmes, batches, subjects, students, faculty and rooms. Examination planning then selects batches, calendar dates and shifts. Automatic scheduling creates traceable runs and conflict records. Published examinations feed eligibility, seating, invigilation and attendance. Import and audit tables make administrative actions reviewable.

Measure	Current design
Database engine	MySQL or MariaDB using InnoDB
Tables	35
Declared foreign keys	73
Canonical installation files	database/schema.sql then database/seed.sql
Primary scheduling scope	School, programme, batch, semester, section and selected written subjects
Major operational outputs	Date sheet, seating plan, invigilation duties, attendance and reports

System Data Flow

1. Create schools and departments, then programmes with their duration.

2. Create admission batches for each programme and enrol students into the correct batch.

3. Create reusable courses and map them to a programme and batch through programme_courses.

4. Create an exam cycle, select eligible batches, configure shifts, dates, holidays and rules.

5. Validate and generate a scheduling run; review conflicts before creating examinations.

6. Attach programme and batch cohorts to each examination and determine student eligibility.

7. Generate versioned seating, assign invigilation, record attendance and retain the audit history.

2 Relational Design Concepts

Term	Meaning in this project
Entity	A real system concept represented by a table, such as students or exam_cycles.
Attribute	A property represented by a column, such as roll_number or exam_date.
Primary key	A value that uniquely identifies a row. Most tables use id.
Foreign key	A column that must reference an existing row in another table.
Unique key	A rule preventing duplicate business data or duplicate combinations.
Junction table	A table resolving a many-to-many relationship, such as exam_cycle_batches.

Term	Meaning in this project
Index	A database structure that accelerates filtering and joining.
Transaction	A group of writes that either all commit or all roll back.

Relationship Types Used

- One to many: one school offers many programmes.
- Many to many through a junction: an exam cycle selects many batches and a batch may appear in many cycles.
- Identifying dependency: room seats cannot exist without a room.
- Optional historical reference: a generated examination can remain after its scheduling run reference is set to NULL.

3 Complete Table Dictionary

Every table in the current schema is listed below. The purpose column is the simplest sentence to use when the jury asks what the table stores.

Security and governance

Table	Purpose	Primary key	Main relationship
roles	Role definitions and permission JSON	id	Parent of users
users	Authenticated application accounts	id	References roles

Academic master

Table	Purpose	Primary key	Main relationship
schools	University schools	id	Parent of departments, programmes and faculty
departments	Departments within a school	id	References schools
programmes	Branches and academic programmes	id	References schools and departments
batches	Admission cohorts such as UCS 2023-2027	id	References programmes
courses	Reusable subject master	id	Referenced by batch curriculum and examinations
programme_courses	Batch-wise curriculum by semester and section	id	References programme, batch and course

People

Table	Purpose	Primary key	Main relationship
students	Student identity and academic placement	id	References programme and batch
faculty	Faculty master and organisational placement	id	References school and department

Infrastructure

Table	Purpose	Primary key	Main relationship
rooms	Examination rooms and capacities	id	Parent of room seats
room_seats	Visual seat coordinates and enabled state	id	References rooms; cascades on room deletion

Exam planning

Table	Purpose	Primary key	Main relationship
exam_cycles	An examination window for one academic session	id	Created by a user
exam_cycle_batches	Batches selected for an exam cycle	cycle_id + batch_id	Junction between cycles and batches
exam_shifts	Morning, evening or custom sessions	id	References exam cycles
exam_calendar_dates	Enabled dates, Sundays and holidays	id	References exam cycles

Automatic scheduling

Table	Purpose	Primary key	Main relationship
scheduling_rules	School-specific gap and priority configuration	id	References cycle, school and user
scheduling_runs	Validation, generation and publication attempts	id	References cycle, school and users
scheduling_run_items	Selected curriculum paper and assigned slot	id	References run, curriculum, shift and exam
scheduling_conflicts	Blocking errors and warnings detected by validation	id	References scheduling run

Date sheet

Table	Purpose	Primary key	Main relationship
examinations	Scheduled written paper on a date and shift	id	References cycle, shift, course and run
examination_cohorts	Programme, batch, semester and section rows for a paper	id	References examination, programme and batch
exam_eligibility	Student-level eligibility for a paper	id	References examination and student

Seating

Table	Purpose	Primary key	Main relationship
seating_allocations	Versioned seating generation for a session	id	References cycle, shift and generator
seating_assignments	Student to room and seat assignment	id	References allocation, exam, room, seat and student
seating_unallocated	Students that could not receive a seat	id	References allocation, exam and student

Invigilation

Table	Purpose	Primary key	Main relationship
faculty_availability	Faculty availability by date and shift	id	References faculty and shift
invigilation_allocations	Room duty assigned to faculty	id	References cycle, shift, room, faculty and user
replacement_requests	Replacement workflow for an invigilation duty	id	References duty, users and replacement faculty

Attendance

Table	Purpose	Primary key	Main relationship
attendance	Student attendance for an examination	id	References seating, examination, room, student and users

Imports

Table	Purpose	Primary key	Main relationship
import_batches	Uploaded bulk file and processing summary	id	References uploader
import_rows	Normalized staged rows before commit	id	References import batch
import_errors	Row and column validation errors	id	References import batch

Governance

Table	Purpose	Primary key	Main relationship
audit_logs	Immutable record of important actions	id	References user; entity reference is polymorphic

Configuration

Table	Purpose	Primary key	Main relationship
system_settings	Application-wide typed settings	setting_key	References updating user

4 Core Entity Relationships

Academic hierarchy

```
SCHOOL -> DEPARTMENT -> PROGRAMME -> BATCH -> STUDENT
                    PROGRAMME + BATCH + COURSE -> PROGRAMME_COURSE
```

The programme is the branch, while the batch is the admission cohort. A student belongs to one programme and one batch. A course is a reusable subject master. programme_courses supplies the missing context by stating that a particular course belongs to a particular programme and batch in a specific semester and section.

Why programme and batch must not be merged

UCS describes B.Tech Computer Science and Engineering, but UCS 2023-2027 identifies one admission cohort. Keeping these in separate tables supports multiple active batches, completed batches, lateral-entry durations and curriculum changes between admission years.

Important business uniqueness rules

Rule	Purpose
department school and code	Department code is unique within its school.

Rule	Purpose
batch programme and label	A programme cannot contain duplicate batch labels.
programme batch course semester section	A batch curriculum mapping cannot be duplicated.
room and seat label	A room cannot contain duplicate seat numbers.
cycle and shift sequence	Shift order remains unique inside a cycle.
cycle and calendar date	A date is configured once per cycle.

5 Automatic Scheduling Data Model

Scheduling scope

The generator works school-wise but keeps every selected programme, batch, semester and section as a distinct cohort. Courses do not become one combined branch row. Subject filters affect only the selected curriculum mappings and retain the user's deselection state.

```
EXAM_CYCLE -> SELECTED_BATCHES + CALENDAR_DATES + SHIFTS
PROGRAMME_COURSES -> SCHEDULING_RUN -> RUN_ITEMS -> EXAMINATIONS -> EXAMINATION_COHORTS
```

Validation sequence

1. Confirm selected programmes belong to the chosen school.

2. Resolve the active/current semester for each selected programme batch.

3. Load active batch-wise curriculum and exclude labs or practical subjects.

4. Apply explicit subject selections and priorities.

5. Check enabled dates, holidays, shifts and requested gaps.

6. Evaluate capacity for each cohort rather than imposing a fixed example limit.

7. Store blockers and warnings in scheduling_conflicts.

8. Generate examinations and examination_cohorts only after a valid run is saved.

Gap and holiday logic

A preferred gap is applied when calendar capacity allows it. If a cohort has as many papers as available working days, the generator must use consecutive enabled days rather than fail. Sundays and configured holidays are never selected as examination dates. Different branches may use the same date and shift when they do not represent the same cohort conflict.

6 Seating Invigilation and Attendance

Seating

Rooms contain individually addressable seats. Disabled seats remain part of the visual room layout but are excluded from allocation. Each seating generation creates a versioned seating_allocations row; assignments and failures belong to that version.

Invigilation

Faculty availability is recorded by date and shift. Invigilation duties connect a cycle, shift, room and faculty member. Replacement requests preserve the original duty and the review trail rather than directly overwriting the assignment.

Attendance

Attendance connects the examination, student, room and seating allocation. The examination and student combination is unique, ensuring that one candidate cannot receive multiple final attendance states for one paper.

7 Integrity Normalization and Performance

Normalization

Normal form idea	How the design applies it
First normal form	Columns contain atomic values and repeating records use separate rows.
Second normal form	Junction-table attributes describe the full relationship key.
Third normal form	Schools, programmes, courses, batches and people are stored once and referenced by keys.

Referential actions

- CASCADE is used for owned detail data, including room seats, selected cycle batches, calendar dates, run details and import staging rows.
- SET NULL is used when a historical examination or run item should survive without its former generator reference.
- RESTRICT is the safe default for operational records that require deliberate dependency review before deletion.

Indexes and query performance

The schema indexes common academic filters, date and shift lookups, subject priority, audit entities and import rows. Unique constraints also create indexes. For large deployments, query plans should be reviewed with EXPLAIN before adding new indexes because excessive indexing slows imports and updates.

Transaction requirements

- Commit bulk imports only when all intended valid rows can be persisted.
- Create a scheduling run, its items and resulting examinations atomically.
- Generate seating assignments and the unallocated list in one transaction.
- Delete complex exam-cycle history through a service that removes dependants in a verified order.

8 Database Installation and Maintenance

Fresh installation

1. Create or select an empty MySQL database.

2. Import database/schema.sql.

3. Import database/seed.sql.

4. Set DB_HOST, DB_PORT, DB_DATABASE, DB_USERNAME and DB_PASSWORD in the environment file.

5. Open the health endpoint and confirm database connected.

6. Sign in and verify roles, programmes, batches and curriculum before creating a cycle.

For an existing database, do not blindly re-import the full schema because CREATE TABLE statements are intended for a fresh installation. Back up first and apply a reviewed migration when structural changes are required.

Schema and seed responsibilities

File	Contains	Does not replace
schema.sql	Tables, columns, keys, indexes and constraints	A migration plan for an already populated database
seed.sql	Repeatable baseline roles, masters, curriculum and settings	A backup of operational data created in the UI

Backup practice

- Export both structure and data before deployments or major imports.
- Use dated backup names and retain more than one generation.
- Do not store production credentials or raw backups in the public web directory or Git repository.
- Test restoration into a separate database; an untested backup is not a verified recovery plan.

9 Demonstration and Verification Queries

These read-only queries are suitable for a jury demonstration. Replace numeric IDs only when necessary.

Confirm selected database

```
SELECT DATABASE();
```

Count application tables

```
SELECT COUNT(*) AS total_tables FROM information_schema.tables WHERE table_schema =
DATABASE() AND table_type = 'BASE TABLE';
```

Show programme batches

```
SELECT p.code, p.name, b.label, b.start_year, b.end_year, b.status FROM programmes p JOIN
batches b ON b.programme_id = p.id ORDER BY p.code, b.start_year;
```

Show batch curriculum

```
SELECT p.code, b.label, pc.semester, c.code AS course_code, c.name, pc.subject_priority FROM
programme_courses pc JOIN programmes p ON p.id = pc.programme_id JOIN batches b ON b.id =
pc.batch_id JOIN courses c ON c.id = pc.course_id ORDER BY p.code, b.start_year, pc.semester,
pc.subject_priority;
```

Show scheduled cohorts

```
SELECT e.exam_date, s.name AS shift_name, c.code AS course_code, p.code AS programme_code,
b.label, ec.semester, ec.section FROM examinations e JOIN exam_shifts s ON s.id = e.shift_id
JOIN courses c ON c.id = e.course_id JOIN examination_cohorts ec ON ec.examination_id = e.id
JOIN programmes p ON p.id = ec.programme_id JOIN batches b ON b.id = ec.batch_id ORDER BY
e.exam_date, s.sequence_no, p.code, b.start_year;
```

Check seating duplicates

```
SELECT allocation_id, room_id, seat_id, COUNT(*) AS uses FROM seating_assignments GROUP BY
allocation_id, room_id, seat_id HAVING COUNT(*) > 1;
```

Check latest imports

```
SELECT id, import_type, file_name, total_rows, valid_rows, invalid_rows, status FROM
import_batches ORDER BY id DESC LIMIT 10;
```

Check latest audit actions

```
SELECT created_at, user_id, action, entity_type, entity_id FROM audit_logs ORDER BY id DESC
LIMIT 20;
```

10 Troubleshooting Guide

Symptom	Likely cause	Safe check or correction
Unknown column source_row_number	Existing import_rows table predates the current schema	Inspect SHOW COLUMNS first; apply a targeted migration rather than repeating an invalid rename.
Foreign-key error while deleting	Dependent operational records still reference the parent	Identify child rows from the FK matrix and use the application deletion service or a reviewed transaction.
Only information_schema views appear	phpMyAdmin is displaying the system database	Select the application database and run SELECT DATABASE().
Access denied for CREATE TEMPORARY TABLE	Shared host restricts temporary-table privileges	Use the hosting-compatible seed package or import a seed version that avoids restricted statements.
Login redirects to login	Credentials, password hash, session storage	Confirm database connection, active user, password

Symptom	Likely cause	Safe check or correction
	or cookie configuration is invalid	hash and production session/cookie settings.
No curriculum papers found	The selected programme, batch and semester has no active programme_courses	Verify the exact batch label and curriculum mapping with the batch curriculum query.
Schedule capacity warning	Requested papers and gaps exceed enabled slots for a cohort	Review enabled dates, shifts and preferred gap; do not treat unrelated branches as the same cohort.
Uploaded file cannot be stored	Upload directory is missing or not writable	Verify configured storage path, PHP upload limits and server permissions.

11 Jury Question Bank

Answer the first sentence directly, then add the second sentence only if the jury asks for implementation detail.

Q1. What problem does the database solve?

It provides one consistent source of truth for academic masters, batch-wise curriculum, exam cycles, automatic date-sheet generation, seating, invigilation, attendance, imports and audit history.

Q2. Why is the database relational?

The data contains stable relationships such as school to programme, programme to batch and examination to student. Foreign keys, unique constraints and transactions protect these relationships better than duplicated files or isolated spreadsheets.

Q3. Why are courses and programme_courses separate?

courses stores the reusable subject identity. programme_courses stores where that subject applies: programme, batch, semester, section, credits, duration and priority. This avoids duplicating course names while supporting different curricula.

Q4. How is batch-wise curriculum implemented?

programme_courses contains both programme_id and batch_id. A unique constraint prevents the same course from being mapped twice to the same programme, batch, semester and section.

Q5. Why does batch belong to programme?

A label such as UCS 2023-2027 is meaningful only in the context of UCS. programme_id guarantees that the cohort cannot be confused with a similarly dated cohort in another programme.

Q6. How is programme duration stored?

programmes.duration_semesters stores the total duration. Batch end year can be derived from the admission year and duration, while the explicit batch dates preserve the official cohort label.

Q7. How does the system determine the current semester?

It compares the batch admission period with the examination academic session and maps elapsed academic terms to a semester, bounded by programme duration. Completed batches are excluded from regular scheduling.

Q8. How are laboratory subjects excluded?

The application applies the written-paper policy to course category and naming/type information before creating scheduling items. Only eligible written curriculum records proceed to scheduling.

Q9. Why is examination_cohorts required?

One scheduled course paper may serve one or more programme-batch-semester-section cohorts. This table preserves each cohort separately instead of combining branches into an ambiguous text field.

Q10. Can different branches have papers at the same time?

Yes. Different cohorts may share a date and shift when no defined conflict exists. The schedule is cohort-aware, so unrelated branches do not unnecessarily block each other.

Q11. How are subject priorities used?

programme_courses.subject_priority provides an order from high to low importance. The scheduling rule can place higher-priority written subjects earlier while still respecting dates, gaps and shifts.

Q12. How are holidays represented?

exam_calendar_dates stores each date with enabled and holiday flags. The generator schedules only enabled examination dates and can account for Sundays and explicitly configured holidays.

Q13. What is a scheduling run?

It is a traceable attempt to validate or generate a schedule for a school and cycle. It stores configuration, status, creator, approver and publisher, while run items and conflicts preserve the detailed result.

Q14. Why are scheduling conflicts stored?

Persisted conflicts make validation explainable. The UI can show exactly why a run is blocked or warned, and the audit trail remains available for review.

Q15. How is a generated exam linked back to automation?

examinations.generated_by_run_id references scheduling_runs. If the run is removed, the reference becomes NULL so a valid published examination is not accidentally deleted.

Q16. What prevents duplicate examinations?

A unique constraint covers cycle, course, exam date, shift and category. Cohort uniqueness is enforced separately in examination_cohorts.

Q17. How does seating remain versioned?

seating_allocations stores cycle, date, shift and version number. Assignments belong to a specific version, allowing regeneration without overwriting earlier results.

Q18. What prevents two students from receiving the same seat?

seating_assignments has a unique constraint on allocation, room and seat. It also uniquely constrains allocation and student, preventing duplicate placement.

Q19. How are disabled seats handled?

room_seats stores an enabled flag. The allocation service selects only enabled seats, so blocked or damaged seats remain visible in the layout but are never assigned.

Q20. How is faculty double-booking prevented?

Unique duty and availability constraints use faculty, exam date and shift. The allocation logic checks these records before saving an invigilation duty.

Q21. Why is attendance linked to examination and student?

That pair is the real business identity of an attendance result. A unique constraint ensures one final attendance record per student per examination.

Q22. How do bulk imports work safely?

A file creates an import_batches record. Every row is staged in import_rows, validation problems go to import_errors, and only valid rows are committed within a transaction.

Q23. Why is source_row_number used?

It preserves the original CSV row for understandable error reporting and prevents duplicate staged rows inside one import batch.

Q24. What is referential integrity?

It is the guarantee that a foreign-key value points to an existing parent row. For example, a student cannot reference a programme or batch that does not exist.

Q25. What is the difference between primary, foreign and unique keys?

A primary key identifies a row. A foreign key links it to another table. A unique key prevents duplicate business combinations such as one seat being assigned twice in an allocation.

Q26. What does ON DELETE CASCADE mean?

When a parent is deleted, dependent detail records are deleted automatically. It is used where child data has no independent meaning, such as room seats belonging to a room.

Q27. Why is deletion restricted in many tables?

Historical exam, attendance and allocation data must not become orphaned. Restriction forces the application to review or remove dependent records in a controlled sequence.

Q28. What is normalization in this schema?

The design largely follows third normal form: each table represents one subject, repeating groups are separated, and non-key attributes depend on the table key rather than unrelated data.

Q29. Give an example of avoiding redundancy.

A course name is stored once in courses. programme_courses references it and adds curriculum context instead of copying the name for every batch and semester.

Q30. Where is denormalization used?

Operational snapshot or display fields may be retained where historical output must remain stable. Such fields are intentional and do not replace the authoritative master relationships.

Q31. Why are indexes necessary?

Indexes reduce lookup cost for frequent filters such as programme, batch, semester, section, date and shift. They also support unique business rules.

Q32. What transaction boundaries are important?

Import commit, date-sheet generation, seating generation and multi-table deletion should be transactional. Either every dependent write succeeds or the entire operation rolls back.

Q33. How is role-based access represented?

roles stores permission definitions and users references one role. The application checks permissions before controllers allow operations such as publishing, importing or deleting.

Q34. How does audit logging differ from application logs?

audit_logs records business actions, actor and affected entity. Server logs diagnose technical behaviour; audit records explain who changed business data.

Q35. Why is audit entity_id not a foreign key?

It is a polymorphic reference that may identify different table types. Keeping it independent also preserves the audit record after the original entity is removed.

Q36. How is SQL injection prevented?

Application queries use prepared statements with bound parameters. User input is never concatenated directly into executable SQL.

Q37. How are passwords stored?

Only secure password hashes are stored in users.password_hash. Plain-text passwords must never be written to the database or seed file.

Q38. What is the purpose of schema.sql?

It is the canonical fresh-install definition containing every table, constraint and index required by the application.

Q39. What is the purpose of seed.sql?

It inserts repeatable baseline data such as roles, schools, programmes, initial batches, course curriculum and system settings after schema creation.

Q40. Is seed.sql a backup?

No. It is deterministic starter data. User-created cycles, students, seating and operational records require a database backup or export.

Q41. In what order should a fresh database be installed?

Create/select the target database, import schema.sql first, then import seed.sql, configure the environment credentials, and verify the health endpoint and login.

Q42. How is the hosted database different from local XAMPP?

The application model is the same, but hosted credentials use the provider's database name, username and hostname. Shared hosts may restrict CREATE DATABASE and temporary-table privileges.

Q43. How would you back up the database?

Use phpMyAdmin export or mysqldump with structure and data, verify the file, retain dated copies, and test restoration into a separate database.

Q44. How do you prove the schema and seed are compatible?

The project creates a temporary fresh database from schema.sql, runs seed.sql twice, and verifies expected counts. The repeatability test currently passes.

Q45. What are the main future database improvements?

Potential improvements include formal migrations, automated backup retention, read-only reporting replicas for scale, and more granular permission assignments if institutional roles expand.

Appendix A Complete Foreign Key Matrix

The following matrix is generated from the current schema.sql and contains all 73 declared foreign keys.

Child table	Foreign-key column	Parent table	Parent key	On delete
users	role_id	roles	id	RESTRICT
departments	school_id	schools	id	RESTRICT
programmes	school_id	schools	id	RESTRICT
programmes	department_id	departments	id	RESTRICT
batches	programme_id	programmes	id	RESTRICT
programme_courses	programme_id	programmes	id	RESTRICT
programme_courses	batch_id	batches	id	RESTRICT
programme_courses	course_id	courses	id	RESTRICT
students	programme_id	programmes	id	RESTRICT
students	batch_id	batches	id	RESTRICT
faculty	school_id	schools	id	RESTRICT
faculty	department_id	departments	id	RESTRICT
room_seats	room_id	rooms	id	CASCADE
exam_cycles	created_by	users	id	RESTRICT
exam_cycle_batches	cycle_id	exam_cycles	id	CASCADE
exam_cycle_batches	batch_id	batches	id	RESTRICT
exam_shifts	cycle_id	exam_cycles	id	CASCADE
exam_calendar_dates	cycle_id	exam_cycles	id	CASCADE
scheduling_rules	cycle_id	exam_cycles	id	CASCADE
scheduling_rules	school_id	schools	id	RESTRICT
scheduling_rules	created_by	users	id	RESTRICT
scheduling_runs	cycle_id	exam_cycles	id	CASCADE
scheduling_runs	school_id	schools	id	RESTRICT

Child table	Foreign-key column	Parent table	Parent key	On delete
scheduling_runs	created_by	users	id	RESTRICT
scheduling_runs	approved_by	users	id	RESTRICT
scheduling_runs	published_by	users	id	RESTRICT
examinations	cycle_id	exam_cycles	id	RESTRICT
examinations	shift_id	exam_shifts	id	RESTRICT
examinations	course_id	courses	id	RESTRICT
examinations	generated_by_run_id	scheduling_runs	id	SET NULL
examination_cohorts	examination_id	examinations	id	CASCADE
examination_cohorts	programme_id	programmes	id	RESTRICT
examination_cohorts	batch_id	batches	id	RESTRICT
exam_eligibility	examination_id	examinations	id	CASCADE
exam_eligibility	student_id	students	id	RESTRICT
scheduling_run_items	scheduling_run_id	scheduling_runs	id	CASCADE
scheduling_run_items	examination_id	examinations	id	SET NULL
scheduling_run_items	programme_course_id	programme_courses	id	RESTRICT
scheduling_run_items	shift_id	exam_shifts	id	RESTRICT
scheduling_conflicts	scheduling_run_id	scheduling_runs	id	CASCADE
seating_allocations	cycle_id	exam_cycles	id	RESTRICT
seating_allocations	shift_id	exam_shifts	id	RESTRICT
seating_allocations	generated_by	users	id	RESTRICT
seating_assignments	allocation_id	seating_allocations	id	CASCADE
seating_assignments	examination_id	examinations	id	RESTRICT
seating_assignments	room_id	rooms	id	RESTRICT
seating_assignments	seat_id	room_seats	id	RESTRICT
seating_assignments	student_id	students	id	RESTRICT
seating_unallocated	allocation_id	seating_allocations	id	CASCADE
seating_unallocated	examination_id	examinations	id	RESTRICT
seating_unallocated	student_id	students	id	RESTRICT
faculty_availability	faculty_id	faculty	id	RESTRICT
faculty_availability	shift_id	exam_shifts	id	RESTRICT

Child table	Foreign-key column	Parent table	Parent key	On delete
invigilation_allocations	cycle_id	exam_cycles	id	RESTRICT
invigilation_allocations	shift_id	exam_shifts	id	RESTRICT
invigilation_allocations	room_id	rooms	id	RESTRICT
invigilation_allocations	faculty_id	faculty	id	RESTRICT
invigilation_allocations	assigned_by	users	id	RESTRICT
attendance	allocation_id	seating_allocations	id	RESTRICT
attendance	examination_id	examinations	id	RESTRICT
attendance	room_id	rooms	id	RESTRICT
attendance	student_id	students	id	RESTRICT
attendance	marked_by	users	id	RESTRICT
attendance	corrected_by	users	id	RESTRICT
replacement_requests	duty_id	invigilation_allocations	id	RESTRICT
replacement_requests	requested_by	users	id	RESTRICT
replacement_requests	replacement_faculty_id	faculty	id	RESTRICT
replacement_requests	reviewed_by	users	id	RESTRICT
import_batches	uploaded_by	users	id	RESTRICT
import_errors	import_batch_id	import_batches	id	CASCADE
import_rows	import_batch_id	import_batches	id	CASCADE
audit_logs	user_id	users	id	RESTRICT
system_settings	updated_by	users	id	RESTRICT

Appendix B Presentation Script and Revision Checklist

Two minute database explanation

Our application uses a normalized MySQL relational database with 35 tables and 73 foreign keys. The academic hierarchy starts with school, department and programme. Every programme has admission batches, and students belong to the correct programme and batch. Subjects are stored once in courses, while programme_courses creates a batch-wise curriculum for a semester and section. An exam cycle selects batches, dates and shifts. Automatic scheduling validates written subjects, holidays, gaps and priorities, then stores a scheduling run, its items and any conflicts. Valid items become examinations, and examination_cohorts keeps every branch and batch separate. Those examinations then drive eligibility, seating, invigilation and attendance. Foreign keys, unique constraints, transactions and audit logs protect data consistency and traceability.

Before the jury

- [ ] I can explain the difference between programme, batch, course and programme_course.
- [ ] I can trace one student from programme and batch to examination, seat and attendance.
- [ ] I can explain PK, FK, UK, indexes, normalization and transactions with project examples.
- [ ] I can explain why examinations and examination_cohorts are separate.
- [ ] I can explain CASCADE, SET NULL and RESTRICT deletion behaviour.
- [ ] I can run the demonstration queries without modifying data.
- [ ] I know that schema.sql is for structure and seed.sql is baseline data, not a backup.
- [ ] I have a verified database backup before the demonstration.